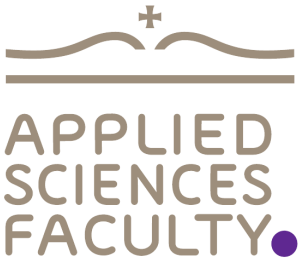
Практичні завдання курсу "Сучасний C++" – чернетка

Олег Фаренюк





1 вересня 2026 р.

Creative Commons - Attribution Share Alike (CC BY-SA)

Зміст

1 Вступ **3**

2 Функції **4**
2.1 Legacy variadic функції та variadic-шаблони 4
2.1.1 Методичні рекомендації 5
2.1.2 Додаткові завдання 5

3 Класи та керування ресурсами **6**
3.1 Раціональні числа 6
3.1.1 Методичні рекомендації 7
3.1.2 Додаткові завдання 7
3.2 my_optional 7
3.2.1 Додаткові завдання 9
3.2.2 Методичні рекомендації 9
3.3 Вектор та масив 11
3.3.1 Динамічний масив 11
3.3.2 Масив фіксованого розміру 14
3.3.3 Додаткові завдання 15
3.4 Smart pointers 15
3.5 Бібліотека лінійної алгебри 15
3.6 128-bit int 16
3.7 Кватерніони 16

4 Узагальнене програмування **17**
4.1 Шаблонні алгоритми 17
4.1.1 Додаткові завдання 18

5 ООП **19**
5.1 Хрестики-нулики 19

Розділ 1

Вступ

У цьому документі описані головні завдання курсу "Сучасний C++". Вони покладаються на теоретичні знання, які обговорюються на лекціях – тому тут свідомо не конкретизують подробиці, для засвоєння яких – і щоб задуматися над якими, запропоновано ці завдання. Тому, зазвичай, більша деталізація вимог не запланована.

Цей документ відносно сирий. Якщо бачите помилки або неузгодженості – пишіть!

Розділ 2

Функції

2.1 Legacy variadic функції та variadic-шаблони

Завдання: написати дві функції чи сімейства перевантажених функцій[1], аналогічні за функціоналом printf() із стандартної бібліотеки C:

- `my_printf`, яка використовує специфікатори, аналогічні std::printf(), та покладається на функції із довільною кількістю аргументів.
- `my_print` – її безпечний щодо переданих типів варіант, виконаний за допомогою variadic templates.
  - ◇ Бажана її поведінка має бути схожою на засоби Python: `https://docs.python.org/3/library/stdtypes.html#str.format`, std::format: `https://en.cppreference.com/w/cpp/utility/format/format`, чи arg() із QString: `https://doc.qt.io/qt-6/qstring.html#arg`.

Написати програму, яка використовує ці функції для виводу текстів та тестує всі реалізовані можливості.

Для `my_printf` обов'язково реалізуйте специфікатори для виведення:

- символу "%",
- змінних типу int,
- double,
- C-string,

[1]На розсуд розробників – але в межах рекомендацій щодо стилю коду C++, які ми вивчали.

- інші – на ваш розсуд.

Для `my_print` – реалізуйте універсальний специфікатор, який вказує: "тут слід вивести черговий аргумент".

Чи реалізовувати специфікатори формату (такі як ширина виводу, точність тощо) – на ваш розсуд.

Код слід оформити у вигляді бібліотеки з хедерів та cpp файлів та програми із демонстрацією використання всіх основних можливостей.

Для компіляції використовувати CMake.

І CMakeLists.txt і бібліотека мають бути стандартними та кросплатформовими.

2.1.1 Методичні рекомендації

- Для здійснення безпосередньо виводу можна використовувати iostream та std::cout.

  ◇ Може бути також цікавим скористатися засобами типу функції сімейства itoa()/atoi() та більш низькорівневим вводом виводом – fwrite абощо.

- Перш ніж приступати до реалізації – познайомтеся із засобами, що служитимуть вам натхненням – std::printf, std::format тощо.

2.1.2 Додаткові завдання

- Порівняйте швидкодію вашої реалізації із стандартними та запропонуйте пояснення виявленим відмінностям.

Розділ 3

Класи та керування ресурсами

3.1 Раціональні числа

Створіть клас роботи із раціональними числами – `ratio_t`.

Ціль: практика коректного перевантаження операторів C++, аналіз компромісів між продуктивністю, коректністю та зручністю.

Повинен реалізовувати:

- всі стандартні арифметичні операції за допомогою перевантаження операторів[1],
- оператори вводу-виводу, симетричних між собою – оператор вводу може прочитати виведене оператором виводу,
- підмножину стандартних математичних функцій C, які мають сенс для раціональних чисел.
  - ◇ Приклад: функція піднесення до цілого степеня має сенс – результатом є теж раціональне число, sin – ні, оскільки результат в раціональних числах представити важко і він не дуже корисний.
  - ◇ Кількість – на ваш розсуд, але не менше трьох різних.
- Конвертування із double у ваш клас – на ваш розсуд.

Потрібними є також тести всіх засобів вашого класу і заміри продуктивності.

[1] Виконаного згідно рекомендацій до стилю, що ми вивчали.

3.1.1 Методичні рекомендації

- Рекомендований дизайн – зберігати раціональні числа (дроби) у "зведеній", однозначній, формі:
  - ◇ дріб завжди зберігається скороченим,
  - ◇ знаменник завжди додатній, (знак дробу визначається чисельником),
  - ◇ якщо знаменник 0 – чисельник завжди 1,
  - ◇ пара $+\frac{1}{0}$ та $-\frac{1}{0}$ – аналог Inf для double,
  - ◇ $\frac{0}{0}$ – аналог NaN для double.
- Реалізація повинна відповідати прийнятим в C++ паттернам, зокрема – арифметичні оператори `@` варто реалізувати як незалежні функції, за допомогою операторів `@=`, які є членами класу.

3.1.2 Додаткові завдання

- Спробуйте реалізувати оптимізацію з використанням SIMD – за допомогою асемблерних вставок чи інтринсик. Порівняйте швидкодію вашої реалізації із стандартними та запропонуйте пояснення виявленим відмінностям. Використання цієї оптимізації повинно керуватися за допомогою умовної компіляції – слід мати можливість їх вимикати вручну, і вмикатися вони повинні лише для компіляції під підтримувані архітектури.

3.2 Керування часом життя на прикладі my_optional

Завдання: написати свій повноцінний аналог шаблону класу std::optional – `my_optional`.

- Об'єкт цього класу можуть зберігати значення або не зберігати нічого.
- Динамічної алокації не відбувається.
- Час життя об'єкта, що зберігається в `my_optional` має коректно закінчуватися.

Необхідно підтримувати наступний функціонал:

- Конструктори:
  - ◇ Конструктор за замовчуванням;
  - ◇ Конструктори копіювання та переміщення.
  - ◇ Конструктор, що створює об'єкт із аргументів конструктора типу, який зберігається в `my_optional`, з таким прототипом:

    ```
    template <typename... Ts> my_optional(std::in_place_t,
                                          Ts&&... args).
    ```

- Деструктор.
- Оператори присвоєння та присвоєння із переміщенням.
- Оператори порівняння: `<`, `<=`, `>`, `>=`, `==`, `!=`, можливо, з використанням spaceship-оператора `<=>`.
- Метод reset(), який робить об'єкт порожнім.
- Метод emplace(), який заміщає об'єкт, що зберігається в `my_optional`:

  ```
  template <typename... Ts> void emplace(Ts&&... args).
  ```

- Метод swap().
- Оператори `const T* operator->() const` та `T* operator->()`, які повертають вказівник на об'єкт, що зберігається в `my_optional` або кидають виключення типу my_bad_optional_access, якщо такий об'єкт відсутній.
- Оператори `const T& operator*() const` та `T& operator*()` – які повертають посилання на об'єкт, що зберігається в `my_optional` або кидають виключення, якщо такий об'єкт відсутній.
- Метод value() із такою ж поведінкою.
- Метод value_or(def), який повертає вміст об'єкту, якщо `my_optional` не порожній і значення def – якщо порожній.

- Метод and_then(func), який повертає результати роботи функції чи іншого callable func до вмісту об'єкту, запакований в `my_optional`, якщо вхідний об'єкт не порожній, або порожній `my_optional` із типом, як у результату func[2].

- Explicit operator bool(), із природною поведінкою.

- Має включати синонім до типу, який зберігає, із іменем value_type.

Реалізація має бути достатньо оптимальною та безпечною щодо виключень.

3.2.1 Додаткові завдання

- Порівняйте швидкодію вашої реалізації із стандартними та запропонуйте пояснення виявленим відмінностям.

3.2.2 Методичні рекомендації

- Хоча для цього завдання можна скористатися підходом із union:

```
template <typename T>
class my_optional {
..............
    union {
        char dummy_m;
        T val_m;
    };
..............
};
```

  однак, оскільки його головною ціллю є знайомство із керуванням ресурсами, зокрема – пам'яттю, яке, у свою чергу, включає врахування вирівнювання, краще скористатися сирою пам'яттю – або реалізувати обома способами.

◇ Зауважте, член dummy_m необхідний, щоб об'єкт типу T не ініціалізувався передчасно.

[2]Тут поведінка, заради спрощення, трішки відрізняється від стандартної – в std:optional є два дещо різних методи, transform() і and_then(), поведінка нашого ближча до стандартного transform().

◇ Якщо таки вирішите скористатися union – зверніть увагу на правила роботи із нетривіальними членами об'єднань: `https://en.cppreference.com/w/cpp/language/union`.

◇ Щоб виділити правильно вирівняну пам'ять, скористайтеся:

```
alignas(T) unsigned char buff_m[sizeof(T)]; //C++11--C++17
alignas(T) std::byte buff_m[sizeof(T)]; //C++20
```

Такий підхід вважається застарілим і не рекомендується[3]:

```
std::aligned_storage_t<sizeof(T), alignof(T)> t_buff;
```

- Пам'ятайте, використовувати динамічну пам'ять тут заборонено!
- Також, заборонено створювати об'єкт типу T без явного запиту – порожній my_optional не має містити створеного об'єкту.
- Для створення та знищення об'єктів у виділеному буфері, можна скористатися placement new та явним викликом деструктора:

```
    new ( buff_m ) { ..... constructor args .....};
    T* obj_ptr = std::launder(reinterpret_cast<T*>(buff_m));
 ............
    obj_ptr->~T();
```

Але можна і допоміжними функціями: std::construct_at[4] std::destroy_at[5] – вони є лише синтаксичним цукром для placement new та виклику деструктора:

```
    std::construct_at(obj_ptr, ..... constructor args .....);
    obj_ptr = std::launder(reinterpret_cast<T*>(buff_m));
    std::destroy_at(obj_ptr);
```

- Функція std::launder(), використана в попередньому пункту, потрібна, щоб повідомити компілятору – об'єкт існує в межах сирого буферу, або замінив собою інший об'єкт цього типу або його предкового типу. Однак, об'єкт має бути створеним якимось іншим чином. Детальніше див. презентацію cpp1_lecture16_casts.pdf, розділ про час життя об'єктів.

[3] `https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p1413r3.pdf`.

[4] C++20, `https://en.cppreference.com/w/cpp/memory/construct_at`. Зверніть увагу, тип об'єкта, який вона створює, визначається типом, на який вказує перший аргумент.

[5] C++17, з C++20 вміє працювати з масивами, див. `https://en.cppreference.com/w/cpp/memory/destroy_at`.

  - ◇ Функції std::start_lifetime_as() та std::start_lifetime_as_array() з C++23 повідомляють інтерпретувати масив байт як об'єкт чи масив об'єктів – розпочинає час життя об'єкту.

- Тут є спокуса створити шаблонний конструктор, який приймає rvalue-reference – і хоча це технічно можливо, не варто цього робити – перевантаження universal reference-функцій це погана ідея. Якщо ж є велике бажання – за порадами див., наприклад: `https://rules.sonarsource.com/cpp/RSPEC-6458` – "Perfect forwarding constructors should be constrained".

3.3 Вектор та масив

Завдання:

- Написати свій повноцінний аналог std::vector, динамічний масив `my_vector`.
- Написати свій аналог std::array, масиву фіксованого розміру, який **не використовує динамічного виділення пам'яті** – `my_array`.

3.3.1 Динамічний масив

Шаблон класу `my_vector`:

- повинен бути коректним,
- надавати гарантії щодо виключень,
- бути достатньо швидким,
- зокрема, відтворюватиме логіку збільшення буферу із std::vector,
  - ◇ хоча використовувати коефіцієнт 2 – не обов'язково,
- Не створюватиме зайві порожні об'єкти і не "затягуватиме" час життя об'єктів, вже видалених із вектора.

Шаблону передається тип елементів[6]:

```
template< typename T>
class my_vector;
```

[6]Опціональні додаткові аргументи із значеннями за замовчуванням дозволені.

В процесі реалізації не потрібно реалізовувати окремі сутності алокаторів, що є в стандартному std::vector, хоча подібне структурування у коді може бути корисним.

Організація проекту – стандартна, my_vector.h (або .hpp) файл[7], файли прикладів/тестів і CMakeLists.txt для їх компіляції.

Ваші шаблони класів повинні бути достатньо щільно покриті тестами (самописними чи із використанням gtest).

Обов'язково потрібно перевіряти коректність збереження в контейнерах фундаментальних типів (наприклад, int), простих класів (std::complex чи ваших ratio_t), складних класів – std::vector, std::string та самого себе – `my_vector<my_vector<int>>` тощо.

Випробуйте також із стандартними алгоритмами, типу std::copy, std::transform тощо.

Необхідний функціонал:

- Конструктори:
  - ◇ Конструктор за замовчуванням;
  - ◇ конструктор, що створює вектор із n копій переданого елемента;
  - ◇ конструктор, який копіює інтервал, заданий ітераторами;
  - ◇ конструктор із списку ініціалізації.
  - ◇ Конструктори копіювання та переміщення.
- Деструктор.
- Оператори присвоєння та присвоєння із переміщенням.
- Оператор звертання за індексом, operator[] без перевірки виходу за межі (можливо, крім відладочної версії),
  - ◇ метод at(), який кидатиме виключення при виході за межі, front() та back().
- Ітератори, сумісні із адаптерами стандартної бібліотеки та методи для роботи із ними (begin(), end(), cbegin(), cend(), rbegin(), rend(), rcbegin(), rcend()).

[7]Файли .cpp у цьому завданні не видаються потрібними – все рівно тіла шаблонів мають бути в файлах заголовків, але, за потреби – назвіть їх так, як і заголовки, з розширенням .cpp.

- ◇ Зокрема, ваш контейнер має бути придатним до використання із адаптерами типу std::back_inserter.

- Методи: is_empty(), (стандартна назва таки невдала), size(), reserve(), capacity(), shrink_to_fit().

- Метод swap().

- clear(), resize().

- insert(), у двох варіантах, перший приймає ітератор, куди вставляти і значення, другий – ітератор, куди вставляти та пару ітераторів – звідки брати. Повинна повертати ітератор на перший вставлений елемент.

- erase() – аналогічно, в двох варіантах, для видалення одного елементу і інтервалу. Повертає ітератор на перший залишений елемент.

- pop_back() – викидає останній елемент, (не повертаючи його), push_back() та emplace_back().

- Оператори `==`, `!=`, `<`, `<=`, `>`, `>=`, порівняння лексикографічне. Для реалізації можна використовувати spaceship-оператор.

Методичні рекомендації

- Завдання нетривіальне – з першого разу виходить рідко.

- Перш ніж переходити до реалізації, познайомтеся із описом `https://en.cppreference.com/w/cpp/container/vector`.

- Не знехтуйте "вкладеними" випробуваннями – `my_vector<my_vector<...>>` – вони виявляють цікаві типові проблеми.

- Потрібно, щоб коректно працював код виду v.push_back(v.back()), навіть якщо при цьому буде відбуватися реалокація.

- ◇ Однак, песимізація – приймати аргумент завжди за значенням[8], тут теж погана ідея.

[8]Щоб під час реалокації посилання не перетворилося на висяче (dangling).

  - ◇ Зручним може бути рішення, яке використовується в стандартних бібліотеках наших компіляторів – *у випадку реалокації, спершу копіювати у новий блок пам'яті елемент*, який додається, а тоді вже решту елементів вектора. Але воно дещо ускладнює декомпозицію коду.

- Подробиці тривалий час обговорювалися на лекціях.

- Обов'язковим є тестування за допомогою valgrind та санітайзерів – вони виявлять багато проблем.

- Обов'язково скористайтеся статичними аналізаторами, наприклад – PVS-Studio.

3.3.2 Масив фіксованого розміру

Шаблону передається тип елементів і розмір:

```
template< typename T, std::size_t N>
class my_array;
```

Релевантні вимоги до `my_array` – ті ж, що і для `my_vector`.

Оскільки масив фіксованого, відомого на момент компіляції, розміру, це ОК, що всі його об'єкти створюються під час створення масиву – явного керування часом життя, як для вектора, не потрібно.

Організація проекту – стандартна, my_array.h (або .hpp) файл[9], файли прикладів/тестів і CMakeLists.txt для їх компіляції.

Ваші шаблони класів повинні бути достатньо щільно покриті тестами (самописними чи із використанням gtest).

Обов'язково потрібно перевіряти коректність збереження в контейнерах фундаментальних типів (наприклад, int), простих класів (std::complex чи ваших ratio_t), складних класів – std::vector, std::string та самого себе – `my_array<my_array<int, 5>, 5>` тощо.

Випробуйте також із стандартними алгоритмами, типу std::copy, std::transform тощо.

Методичні рекомендації

- На відміну від вектора, це просте завдання.

[9] Файли .cpp у цьому завданні не видаються потрібними – все рівно тіла шаблонів мають бути в файлах заголовків, але, за потреби – назвіть їх так, як і заголовки, з розширенням .cpp.

- Перш ніж переходити до реалізації, познайомтеся із описом: `https://en.cppreference.com/w/cpp/container/array`.

- Оскільки my_array відтворює поведінку вбудованих масивів, це ОК, що всі об'єкти створюються наперед, тому для збереження даних варто використати член даних, який є вбудованим масивом.

- ◇ Також, не потрібно явно керувати часом життя об'єктів, що зберігаються – звичайне присвоєння в багатьох випадках працюватиме добре.

- Обов'язковим є тестування за допомогою valgrind та санітайзерів – вони виявлять багато проблем.

- Обов'язково скористайтеся статичними аналізаторами, наприклад – PVS-Studio.

3.3.3 Додаткові завдання

- Порівняйте швидкодію вашої реалізації із стандартними та запропонуйте пояснення виявленим відмінностям.

3.4 Smart pointers

Написати свої аналоги розумних вказівників: `my_unique_ptr`, `my_shared_ptr`, `my_weak_ptr`.

Бажаною є підтримка користувацьких делетерів.

3.5 Додаткова робота: Бібліотека лінійної алгебри

Завдання: створити бібліотеку лінійної алгебри. Покращити її ефективність за допомогою розпаралелення, SIMD, cache-friendliness.

- Бібліотека повинна підтримувати вектори та матриці.

- Допускається як використання єдиного класу прямокутних матриць, із typedef-синонімами для векторів і квадратних матриць, так і окремий клас для векторів і окремий для матриць (можна – лише прямокутних).

- Бажаною є підтримка як статичних, так і динамічних матриці.
- Слід підтримувати наступні операції: додавання, віднімання, множення матриць та векторів, знаходження детермінанту та оберненої матриці.
- Бажаними є також інструменти знаходження власних значень та власних векторів.
- Можна обмежитися підтримкою лише щільних матриць.
- Обов'язковими є тести та "бенчмарки" – засоби для перевірки продуктивності.

3.6 Додаткова робота: клас 128-бітного знакового цілого числа

Завдання: створити високоефективний клас роботи із 128-бітними числами.

Повинен містити всі арифметичні операції та підмножину стандартних математичних функцій C. Вимоги аналогічні до завдання 3.1.

Потрібно реалізувати оптимізації за допомогою SIMD та інших низькорівневих оптимізацій, які керуються умовною компіляцією – щоб вмикати їх лише для підтримуваних платформ. Потрібними є також тести і заміри продуктивності.

Ціль: окрім практики коректного перевантаження операторів C++, дозволяє попрактикуватися у гнучкому використанні платформо-специфічних оптимізацій.

3.7 Додаткова робота: клас кватерніонів

Завдання: створити клас роботи із кватерніонами.

Повинен містити всі арифметичні операції та підмножину стандартних математичних функцій C. Потрібними є також тести. Вимоги аналогічні до завдання 3.1.

В принципі, це – найпростіше практичне завдання для практики коректного перевантаження операторів C++ та іншого суміжного синтаксичного цукру.

Розділ 4

Узагальнене програмування

4.1 Шаблонні алгоритми

Завдання: реалізувати міні-бібліотеку алгоритмів, аналогічних стандартним алгоритмам C++:

- my_copy – аналог std::copy.
  - ◇ Повинна враховувати різницю між POD і не POD і оптимізувати копіювання POD.
  - ◇ Врахуйте, що POD-елементи не обов'язково йдуть один за одним – перевіряйте також на контейнерах типу std::list[1].
  - ◇ Може містити додаткові оптимізації, наприклад, із врахуванням гарантій щодо виключень методів класів, об'єкти яких копіюються.
- my_transform – аналог std::transform, включаючи варіанти із двома і трьома інтервалами,
- my_map_reduce – який реалізовує концепцію map-reduce, див. `https://en.wikipedia.org/wiki/MapReduce`.
- В реалізації заборонено використовувати стандартні алгоритми.

За допомогою вашого my_map_reduce слід реалізувати:

- обчислення суми квадратів елементів певного масиву,

[1] Див. `https://en.cppreference.com/w/cpp/iterator/contiguous_iterator`, C++20.

- пошук найдовшої стрічки у масиві,
- підрахунок, скільки раз кожне слово трапилося у файлах із списку, (заради простоти вважаємо, що файли текстові, обмежуємося кодуванням ASCII).

Перші два завдання – тривіальні вправи, які могли б бути у підручнику для цієї бібліотеки.

Наявність тестів – обов'язкова!

4.1.1 Додаткові завдання

- Коректні оптимізації, (зокрема, без використання паралельності), вітаються.
- Порівняйте швидкодію вашої реалізації із стандартними та запропонуйте пояснення виявленим відмінностям.

Розділ 5

ООП

5.1 Хрестики-нулики

Завдання: напишіть гру хрестики-нулики або аналогічну.

- Гра повинна обов'язково дозволяти, щоб за кожну із сторін могла грати людина та, мінімум, два різних види AI (штучного інтелекту комп'ютерного гравця).
- Важливо, що додавання нових видів AI повинне бути простим!
- Сам AI може бути як завгодно примітивним – він не є ціллю завдання. Однак – чим AI потужніший, тим краще.
- Інтерфейс може бути консольним, GUI або мережевим.
- Корисним буде варіант, що вміє перемикатися між різними UI.
- Перевірка невірного вводу користувачів, зокрема – неможливих ходів, обов'язкова.

Приклад, як би могла виглядати[1]:

```
h - human, cr - computer, using random, cb -- computer, using binary tree.
First player: cr
Second player: cb
|_|_|_|
|_|_|_|
|_|_|_|
```

[1]Не обов'язково дотримуватися суворо

```
|_|_|_|
|_|_|_|
|X|_|_|


|_|O|_|
|_|_|_|
|X|_|_|


|_|O|_|
|_|_|_|
|X|_|X|


|O|O|_|
|_|_|_|
|X|_|X|


|O|O|_|
|_|_|X|
|X|_|X|


|O|O|O|
|_|_|X|
|X|_|X|


Second player won!
```